own constraints requires ORM features beyond the scope of this book (see: https://docs.djangoproject.com/en/dev/ref/models/constraints/ for more information).

5.4.4 Object names

Inside the Django Admin, the name of your `Model` is directly derived from the `Model` class name. For example, the `Musician` class appears as *musician*, *musicians*, *Musician*, and *Musicians*, depending on the context. Take a look at the Musician Listing page; on this page, you will find *Select musician to change* in the title and *Musicians* in the navigation breadcrumbs.

If your `Model` class consists of multiple words formatted in camel case, Django parses it into spearate words, capitalizing when necessary and appending the letter *s* to pluralize. For example, the `MusicianClassified` object is shown as *Musician classifieds* in its plural form. If you're not happy with how this is done, the `verbose_name` and `verbose_name_plural` attributes give you control. These two `Meta` attributes are available on the class, but I've only ever seen them used by the Django Admin.

A `verbose_name` containing capitals appears that way everywhere. If the attribute is lowercase, it is capitalized where Django sees fit. I don't like the mixed-case class name in the navigation bar, so I often set `verbose_name` to use the capitalization I want. Without `verbose_name`, `MusicianClassified` appears as *Musician classifieds* in the nav. I set `verbose_name` to `Musician Classified` to get *Musician Classifieds* in the nav. The downside is the title becomes *Select Musician Classified to change*. Pick your preference.

The most common use of `verbose_name_plural` is for words that don't pluralize in normal ways or that are already plural. Consider a model named `Box`:

```
class Box(models.Model):
    # field declarations
    ...

    class Meta:
        verbose_name_plural = "boxes"
```

Use of `verbose_name_plural` alone, or along with `verbose_name`, gets your `Model` names showing properly. My mother, the retired kindergarten teacher, thanks you for your proper grammar.

5.5 Exercises

1. Add search capabilities to the `BandAdmin` class.
2. Add default sorting by name to the `Band`, `Venue`, and `Room` classes.
3. Create `admin.ModelAdmin` classes for the `Venue` and `RoomModel` classes, including custom-linked column methods, like with `Musician` and `Band`, and searchable name fields.
4. Add a Members column to the Band Listing page, containing multiple links, one to each `Musician` in the `Band`. Note that appending to a string makes it unsafe again. Use `mark_safe` to fix that.